

Practical 06 - Named Pipes and POSIX Signal Handling

Objective

1. To create a client-server application using Named Pipes (FIFOs).
2. To study communication between multiple clients and a server using FIFOs.
3. To implement POSIX signal handling for SIGINT, SIGTERM, and SIGUSR1.
4. To demonstrate asynchronous event handling using signal handlers.

Part 1 - Client-Server Communication Using Named Pipes

Files

- `fifo_server.c` - Server program
- `fifo_client.c` - Client program

Working

The server creates a common request FIFO:

```
/tmp/osp_request_fifo
```

Each client creates a unique response FIFO using its Process ID:

```
/tmp/osp_response_fifo_PID
```

Communication flow:

Client -> Request FIFO -> Server -> Client Response FIFO -> Client

The client sends its PID and message to the server. The server processes the message and sends a response to the corresponding client FIFO.

Compilation

```
gcc fifo_server.c -o fifo_server
gcc fifo_client.c -o fifo_client
```

Codes-

```
#include <stdio.h>
```

```
#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <fcntl.h>

#include <sys/stat.h>


#define REQUEST_FIFO "/tmp/ossdp_request_fifo"

#define BUFFER_SIZE 256


typedef struct
{
    pid_t client_pid;
    char message[BUFFER_SIZE];
} Request;


int main()
{
    int request_fd;
    int response_fd;
    Request request;
    char response_fifo[100];
    char response[BUFFER_SIZE];


    request.client_pid = getpid();


    snprintf(response_fifo, sizeof(response_fifo),
              "/tmp/ossdp_response_fifo_%d",
              request.client_pid);
```

```
mkfifo(response_fifo, 0666);
```

```
printf("=====\n");
```

```
printf("    FIFO CLIENT\n");
```

```
printf("=====\n");
```

```
printf("Enter message: ");
```

```
fflush(stdout);
```

```
fgets(request.message, BUFFER_SIZE, stdin);
```

```
request.message[strcspn(request.message, "\n")] = '\0';
```

```
request_fd = open(REQUEST_FIFO, O_WRONLY);
```

```
if (request_fd == -1)
```

```
{
```

```
    perror("Could not connect to server");
```

```
    unlink(response_fifo);
```

```
    return 1;
```

```
}
```

```
write(request_fd, &request, sizeof(Request));
```

```
close(request_fd);
```

```
printf("Message sent to server.\n");
```

```
printf("Waiting for response...\n");

response_fd = open(response_fifo, O_RDONLY);

if (response_fd == -1)
{
    perror("Could not open response FIFO");
    unlink(response_fifo);
    return 1;
}

read(response_fd, response, BUFFER_SIZE);

printf("Server Response: %s\n", response);

close(response_fd);
unlink(response_fifo);

return 0;
}
```

```
#include <stdio.h>
```

```
#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <fcntl.h>

#include <sys/stat.h>


#define REQUEST_FIFO "/tmp/ossd_request_fifo"

#define BUFFER_SIZE 256


typedef struct
{
    pid_t client_pid;
    char message[BUFFER_SIZE];
} Request;


int main()
{
    int fd;

    Request request;

    char response_fifo[100];
    char response[BUFFER_SIZE];


    printf("=====\n");
    printf("    FIFO SERVER STARTED\n");
    printf("=====\n");


    mkfifo(REQUEST_FIFO, 0666);
```

```

printf("Waiting for client messages...\n");
printf("Press Ctrl+C to stop the server.\n\n");

fd = open(REQUEST_FIFO, O_RDWR);

if (fd == -1)
{
    perror("open");
    return 1;
}

while (1)
{
    ssize_t bytes = read(fd, &request, sizeof(Request));

    if (bytes > 0)
    {
        printf("Received from Client PID %d: %s\n",
            request.client_pid, request.message);

        snprintf(response, BUFFER_SIZE,
            "Server processed your message: %.220s",
            request.message);

        snprintf(response_fifo, sizeof(response_fifo),
            "/tmp/ossdp_response_fifo_%d",
            request.client_pid);
    }
}

```

```

int response_fd = open(response_fifo, O_WRONLY);

if (response_fd != -1)
{
    write(response_fd, response, strlen(response) + 1);
    close(response_fd);
}
else
{
    perror("Could not open client response FIFO");
}
}

close(fd);
unlink(REQUEST_FIFO);

return 0;
}

```

PRACTICAL 06 - OUTPUT

=====

PART 1 - NAMED PIPE CLIENT-SERVER

SERVER OUTPUT

=====

FIFO SERVER STARTED

=====

Waiting for client messages...

Press Ctrl+C to stop the server.

Received from Client PID 1234: Hello Server

Received from Client PID 1250: Hello from Client 1

Received from Client PID 1260: Hello from Client 2

CLIENT OUTPUT

=====

FIFO CLIENT

=====

Enter message: Hello Server

Message sent to server.

Waiting for response...

Server Response: Server processed your message: Hello Server

MULTIPLE CLIENT ANALYSIS

Multiple clients sent messages to the common request FIFO.

The server received and processed requests sequentially.

Each client received its response through a separate FIFO created using its Process ID.

PART 2 - POSIX SIGNAL HANDLING

```
=====
POSIX SIGNAL HANDLING PROGRAM
=====
```

Process PID: 1500

Press Ctrl+C to send SIGINT.

Use another terminal for SIGUSR1:

kill -USR1 1500

Use another terminal for SIGTERM:

kill -TERM 1500

Program is running...

After pressing Ctrl+C:

[HANDLER] SIGINT received - Ctrl+C pressed.

After executing:

kill -USR1 1500

Output:

[HANDLER] SIGUSR1 received - custom user signal.

After executing:

kill -TERM 1500

Output:

[HANDLER] SIGTERM received - terminating program.

Program terminated safely

Signal handling.c-

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <signal.h>
```

```
#include <unistd.h>
```

```
volatile sig_atomic_t running = 1;
```

```
void signal_handler(int sig)
```

```
{
```

```
    if (sig == SIGINT)
```

```
    {
```

```

        printf("\n[HANDLER] SIGINT received - Ctrl+C pressed.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\n[HANDLER] SIGTERM received - terminating program.\n");
        running = 0;
    }
    else if (sig == SIGUSR1)
    {
        printf("\n[HANDLER] SIGUSR1 received - custom user signal.\n");
    }
}

```

```

int main()
{
    struct sigaction sa;

    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);

    printf("=====\n");
    printf("  POSIX SIGNAL HANDLING PROGRAM\n");
    printf("=====\n");
}

```

```
printf("Process PID: %d\n", getpid());  
printf("Press Ctrl+C to send SIGINT.\n");  
printf("Use another terminal for SIGUSR1:\n");  
printf("kill -USR1 %d\n", getpid());  
printf("Use another terminal for SIGTERM:\n");  
printf("kill -TERM %d\n\n", getpid());
```

```
while (running)  
{  
    printf("Program is running...\n");  
    sleep(3);  
}
```

```
printf("Program terminated safely.\n");
```

```
return 0;  
}
```